

Jacob Yankee
COSC 341
Zhang

- I. Reference
 - A. No references were used
- II. Design
 - A. The remv function needed to be called for the remvdub function
 - B. The atomic function was used for suml as expected
 - C. Helper functions were needed for inde, nele, and primeton
- III. Difficulty
 - A. The hardest part of this assignment was translating the filter function in primeton as P(X) is something that cannot be done in prolog.
- IV. Comments
 - A. I don't have a lot to say about this one, surprisingly.
 - B. I suppose asking for comments on the function is new. I only did that where I felt it was necessary.

Appendix One - Source Code

```
/*Function 1  
Jacob Yankee  
COSC 341  
Zhang*/
```

```
/*Function name: delnth  
Description: Deletes the n-th element of a list*/  
delnth([_|Xs], 1, Xs).  
delnth([X|Xs], N, [X|Ans]) :- N1 is N-1, delnth(Xs, N1, Ans).
```

```
/*Function 2  
Jacob Yankee  
COSC 341  
Zhang*/
```

```
/*Function name: dispnth  
Description: Displayed the n-th element of a list*/  
dispnth([X|_], 1, X).  
dispnth([_|Xs], N, Ans) :- N1 is N-1, dispnth(Xs, N1, Ans).
```

```
/*Function 3  
Jacob Yankee  
COSC 341  
Zhang*/
```

```
/*Function name: remv
Description: Removes elements from a list*/
remv(_, [], []).
remv(A, [X|Xs], Ans) :- A = X, remv(A, Xs, Ans).
remv(A, [X|Xs], [X|Ans]) :- remv(A, Xs, Ans).
```

```
/*Function 4
Jacob Yankee
COSC 341
Zhang*/
```

```
/*Function name: remv
Description: Removes elements from a list
Notes: Function 3, helper for remvdub*/
remv(_, [], []).
remv(A, [X|Xs], Ans) :- A = X, remv(A, Xs, Ans).
remv(A, [X|Xs], [X|Ans]) :- remv(A, Xs, Ans).
```

```
/*Function name: remvdub
Description: Removes duplicate elements from a list
Notes: Does not work without remv*/
remvdub([], []).
remvdub([X|Xs], [X|Ans]) :- remv(X, Xs, Ans2), remvdub(Ans2, Ans).
```

```
/*Function 5
Jacob Yankee
COSC 341
Zhang*/
```

```
/*Function name: maxl
Description: Finds the max integer of an integer list*/
maxl([L], L).
maxl([X, Y|Zs], Ans) :- X < Y, maxl([Y|Zs], Ans).
maxl([X, _|Zs], Ans) :- maxl([X|Zs], Ans).
```

```
/*Function 6
Jacob Yankee
COSC 341
Zhang*/
```

```
/*Function name: suml
Description: Returns the sum of a hybrid integer list
Notes: atomic is required for this function to work*/
```

```
suml([], 0).
suml([X|Xs], Ans) :- suml(X, Ans1), suml(Xs, Ans2), Ans is Ans1 + Ans2.
suml(Y,Y) :- atomic(Y).
```

```
/*Function 7
Jacob Yankee
COSC 341
Zhang*/
```

```
/*Function name: oddths
Description: Returns a list that contains only the odd-th elements of the given list*/
oddths([], []).
oddths([A],[A]).
oddths([X,_|Zs], [X|Ans]) :- oddths(Zs, Ans).
```

```
/*Function 8
Jacob Yankee
COSC 341
Zhang*/
```

```
/*Function name: indehelp
Description: Helper function for inde
Notes: Helper function needs to exist to match the input in the assignment*/
indehelp(_,_,[], []).
indehelp(N,I,[X|Xs],[I|Zs]) :- X = N, I1 is I+1, indehelp(N, I1, Xs, Zs).
indehelp(N,I,[_|Xs],Zs) :- I1 is I+1, indehelp(N, I1, Xs, Zs).
```

```
/*Function name: inde
Description: Returns the index of the occurrence of a given value, starting from 1*/
inde(N,L,Ans) :- indehelp(N, 1, L, Ans).
```

```
/*Function 9
Jacob Yankee
COSC 341
Zhang*/
```

```
/*Function name: nelehelp
Description: Helper function for nele
Notes: The helper needs to exist to match the input in the assignment*/
nelehelp(_,_,[], []).
nelehelp(N,0,[_|Xs],Ans):- nelehelp(N,N,Xs,Ans).
nelehelp(N,R,[X|Xs],[X|Ans2]) :- R > 0, R1 is R-1, nelehelp(N,R1,[X|Xs],Ans2).
```

```
/*Function name: nele
```

Description: Repeats each element in a list n times*/
nele(L,N,Ans):- nelehelp(N,N,L,Ans).

/*Function 10
Jacob Yankee
COSC 341
Zhang*/

/*Function name: app
Description: Append function*/
app([], Ys, Ys).
app([X|Xs], Ys, [X|Zs]) :- app(Xs, Ys, Zs).

/*Function name: nums
Description: Creates a list of numbers*/
nums(1, []).
nums(N, Res) :- N > 1, N1 is N-1, nums(N1, Ans2), app(Ans2, [N], Res).

/*Function name: filter
Description: filter function hardcoded to use isprime
Notes: P(X) is not something that can be done in prolog,
so the isprime function gets hardcoded in where P(X) would be.*/
filter(_, [], []).
filter(P, [X|Xs], [X|Ys]) :- isprime(X), filter(P, Xs, Ys).
filter(P, [_|Xs], Ys) :- filter(P, Xs, Ys).

/*Function name: divisible
Description: Determines if a number is divisible*/
divisible(_, 1).
divisible(N, D) :- D > 1, N mod D \= 0, D1 is D-1, divisible(N, D1).

/*Function name: isprime
Description: Determines if a number is a prime*/
isprime(2).
isprime(N) :- N > 2, N1 is N-1, divisible(N, N1).

/*Function name: primeton
Description: Finds all prime numbers from 2 to n*/
primeton(N, Ans) :- nums(N, Ns), filter(isprime, Ns, Ans).

Appendix Two - Sample Runs

Function 1:

Inputs:

delnth([1,2,[3,4],5],3,X).

delnth([a,bc,d,[e,f,g],hi],2,X).

Outputs:

X = [1,2,5]

X = [a,d,[e,f,g],hi]

Function 2:

Inputs:

dispnth([1,[2,3],4,5],2,X).

dispnth([a,b,[cde],g],3,X).

Outputs:

X = [2,3]

X = [cde]

Function 3:

Inputs:

remv(a, [a,b,a,c],X).

remv(3, [5,3,4,3,3,3,2,1],X).

Outputs:

X = [b,c]

X = [5,4,2,1]

Function 4:

Inputs:

remvdub([a,b,a,c,b,a],X).

remvdub([1,2,1,3,1],X).

Outputs:

X = [a,b,c]

X = [1,2,3]

Function 5:

Inputs:

maxl([1, 3, 2, 5, 4], X).

maxl([4,32,5,7,4,12,1],X).

Outputs:

X = 5

X = 32

Function 6:

Inputs:

suml([1,[2,3],[4],5],X).

suml([[1,2,3],[4,5,6]],X).

Outputs

X = 15

X = 21

Function 7:

Inputs:

oddths([2,4,3,1,5,7,9],X).

oddths([a,b,g,d,h,d,as],X).

Outputs:

X = [2, 3, 5, 9]

X = [a, g, h, as]

Function 8:

Inputs:

inde(1,[1,2,1,1,2,2,1],X).

inde(v,[a,c,v,c,a,v,v],X).

Outputs:

X = [1, 3, 4, 7]

X = [3, 6, 7]

Function 9:

Inputs:

nele([1,3,5],3,X).

nele([1],7,X).

Outputs:

X = [1, 1, 1, 3, 3, 3, 5, 5, 5]

X = [1, 1, 1, 1, 1, 1, 1]

Function 10:

Inputs:

primeton(20, X).

primeton(31,X).

Outputs:

X = [2, 3, 5, 7, 11, 13, 17, 19]

X = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]